

Dual-level Reweighting for Positive-Unlabeled Graph Classification

Junghun Kim
Seoul National University
Seoul, South Korea
bandalg97@snu.ac.kr

Shihyung Park
Seoul National University
Seoul, South Korea
shihyung0416@gmail.com

U Kang
Seoul National University
Seoul, South Korea
ukang@snu.ac.kr

Abstract

How can we accurately classify graphs using only positive and unlabeled instances? This setting commonly arises in real-world tasks. For example, in drug side effect prediction, each drug is represented as a graph based on its chemical structure, and only those observed to cause adverse effects are labeled as positive. The remaining drugs are left unlabeled—not because they are confirmed safe, but because such side effects have not yet been discovered or reported. A challenge is that positive and negative graphs often share similar substructures, making it difficult for models to capture the subtle structural differences that distinguish true positives among unlabeled ones. This difficulty is further exacerbated by the absence of explicit negative labels, which leaves the model without clear guidance on how to separate informative from non-informative patterns. We propose Delta-PU, an accurate method for PU graph classification. Delta-PU performs dual reweighting: hop-level reweighting to emphasize informative substructures during pooling, and graph-level reweighting to suppress ambiguous unlabeled graphs. This dual mechanism allows the model to progressively focus on discriminative patterns while reducing the uncertainty introduced by unlabeled instances. Extensive experiments demonstrate that Delta-PU achieves the state-of-the-art performance in PU graph classification.

CCS Concepts

• Computing methodologies → Supervised learning by classification; • Information systems → Social networks.

Keywords

graph classification, PU learning, graph neural network

ACM Reference Format:

Junghun Kim, Shihyung Park, and U Kang. 2026. Dual-level Reweighting for Positive-Unlabeled Graph Classification. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792251>

1 Introduction

Graph classification is a fundamental task in various real-world applications, including molecular property prediction [68, 82], medical diagnosis [1, 34], etc. In these domains, the relation of examples such as molecules or cells are represented as graphs, where nodes

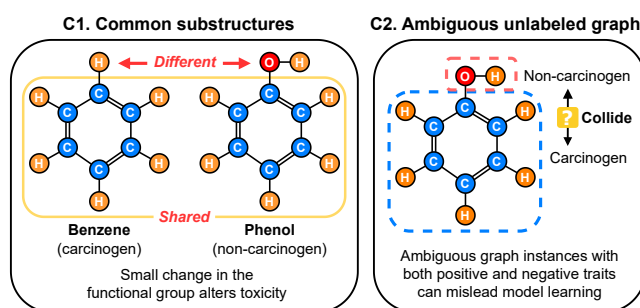


Figure 1: Two main challenges in PU graph classification: a) common substructures across classes obscure subtle discriminative patterns and b) ambiguous unlabeled graphs with both positive and negative traits may mislead learning.

denote atoms or cells and edges represent chemical bonds or cellular interactions between them. In practice, however, obtaining complete labels for graphs is often challenging [36, 74], especially in domains where collecting negative labels is difficult or impractical.

For example, in drug side effect prediction, a drug is labeled as positive if it has been observed to cause any adverse effect, while the majority remain unlabeled—not because they are confirmed safe, but because potential side effects may not yet have been discovered or reported [85]. In web-domain misinformation detection, each article is modeled as a propagation graph—nodes are users who engaged with the article and edges encode retweet/mention relations—and only those reported by fact-checkers as false are labeled positive, while the vast majority remain unlabeled not because they are verified true, but because they have not yet been investigated or reported [5]. These examples represent *positive-unlabeled (PU) graph classification*, where the goal is to learn accurate classifiers given labels only for a subset of positive graphs [83].

In PU graph classification, two main challenges hinder accurate learning. First, many graphs share common substructures regardless of their true labels. For instance, both toxic and non-toxic molecules may contain similar structural motifs, such as aromatic rings [4]. The truly discriminative patterns often reside in small, subtle substructures [47]. However, traditional pooling such as mean or sum pooling [72] treat all node features equally and thus fail to capture these fine-grained differences [76]. Even learning-based pooling approaches [10, 31, 40, 57], while more adaptive, primarily rely on topological semantics and lack explicit supervision to prioritize informative regions, especially in the absence of negative labels. Second, the unlabeled set includes ambiguous instances that exhibit both positive and negative patterns [41, 87]. When treated as fully



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792251>

reliable during training, these graphs can propagate conflicting signals, especially in the absence of confirmed negatives, leading the model to overfit to non-discriminative patterns. The two challenges are summarized in Figure 1.

To address these challenges, we propose DELTA-PU (Dual-level Meta-Reweighting for Positive-Unabeled Graph Classification), an accurate PU graph classification method based on dual-level meta-reweighting. DELTA-PU jointly performs hop-level and graph-level reweighting to enhance both substructure discrimination and instance reliability. At the hop level, DELTA-PU assigns importance weights to representations from different receptive fields. This addresses the challenge where graphs from different classes share large structural components, making it difficult to detect small but critical variations that determine the label, as illustrated in C1 of Figure 1. By adaptively weighting hops, DELTA-PU downplays common global structures and emphasizes informative local variations, enabling finer discrimination of functional groups that are critical for classification. At the graph level, DELTA-PU assigns instance-wise weights to unlabeled graphs based on their generalization performance. This mitigates the effect of ambiguous graphs that exhibit both positive and negative patterns (e.g., C2 in Figure 1), which would otherwise propagate conflicting signals during training. Suppressing such misleading instances leads to cleaner supervision and more robust representation learning. As training progresses, both hop- and graph-level importance weights guide the model toward more accurate predictions. Leveraging these weights, DELTA-PU re-labels the unlabeled graphs and performs a final finetuning stage to further refine performance.

Our contributions are summarized as follows:

- **Method.** We propose DELTA-PU, a dual-level reweighting method that learns to emphasize informative substructures and reliable samples for PU graph classification.
- **Theory.** We provide a theoretical analysis of the time complexity of DELTA-PU to demonstrate its scalability.
- **Experiment.** We evaluate DELTA-PU on multiple benchmark datasets, where it consistently achieves superior performance compared to existing methods.

The symbols used in this paper are summarized in Table 4 (Appendix A.3). Our code and datasets are available at <https://github.com/snudatalab/Delta-PU>.

2 Related Works

2.1 Graph Classification

Graph-structured data has been widely studied in tasks such as node classification [64, 77, 78], link prediction [19, 26, 50], and compression [21]. Among these, graph classification is a fundamental task with widespread applications in chemistry, biology, and social networks [61]. Early approaches use handcrafted graph kernels [60, 65] to measure graph similarity, but they are often limited in scalability and expressiveness. With the rise of deep learning, Graph Neural Networks (GNNs) have become the standard for learning graph-level representations [11, 29]. Modern pipelines separate i) an encoder that maps nodes to embeddings and ii) a readout that aggregates them into a graph vector.

Recent encoders include expressive message-passing networks like GIN [72], which matches the Weisfeiler-Lehman test in discriminative power. Graph Transformers combine local propagation with global attention to capture long-range dependencies [56, 75]. More recently, disentanglement-based encoders for graph classification have appeared [6]. For unsupervised graph classification, SMART [42] adopts graph mixup and prompt tuning. Any of the above encoders can serve as the backbone for our method.

Readout designs include global mean/sum pooling [72], hierarchical pooling [31, 76], and attention-based approaches [10, 13]. Beyond generic readout, recent works learn task-specific substructures [67, 71], highlighting the role of fine-grained regions. Label-limited regimes such as unsupervised and semi-supervised settings have also been explored [63].

However, these readout methods are not designed for the PU setting, where negative labels are unavailable. Without negative supervision, it becomes difficult to distinguish truly discriminative regions from common ones, as the model has no explicit counterexamples to contrast against the positive patterns.

2.2 PU Learning

Positive-Unlabeled (PU) learning addresses the challenge of training a binary classifier when only positive and unlabeled examples are available, without access to confirmed negatives [8, 33]. Early approaches to PU learning typically attempt to either identify likely negative instances from the unlabeled data [12, 37, 39, 49, 66] or reweight the unlabeled examples to act as a mixture of positive and negative data during training [7, 30, 48, 54]. However, they assume flat data representations and overlook structural patterns, limiting their effectiveness in graph classification tasks where class signals emerge from subgraph structures.

PU learning on graph-structured data has recently attracted attention [35, 44, 69, 73, 79, 80, 86]. While these methods leverage the graph structure to improve predictions under PU constraints, they are inherently limited to binary node classification. In contrast, DELTA-PU tackles the more general and challenging setting of graph-level PU classification, where each input is a full graph and labels are sparsely available only for positive instances. We formally define the problem in Problem 1.

PROBLEM 1 (PU GRAPH CLASSIFICATION). *We are given a set $\mathcal{G} = \mathcal{G}_{\text{pos}} \cup \mathcal{G}_{\text{unl}}$ of graphs, where $\mathcal{G}_{\text{pos}}$ is the set of observed positive graphs and $\mathcal{G}_{\text{unl}}$ is the set of unlabeled graphs; $\mathcal{G}_{\text{unl}}$ contains both unobserved positive and negative graphs. For each $G_i \in \mathcal{G}_{\text{pos}}$, a positive label $y_i = 1$ is observed, while for $G_j \in \mathcal{G}_{\text{unl}}$, no label is given. We are also given the true prior π_p , which represent the ground-truth ratio of positive examples among all instances. Then the objective of PU graph classification is to train a binary classifier $f_\theta(\cdot)$ that accurately estimates the probability of a graph $G_j \in \mathcal{G}_{\text{unl}}$ being positive.*

We study the prior-known regime where the positive class prior π_p is given. This is the standard setting in PU learning area [30, 48]. Our method is designed for this regime, and all baselines that require a ground-truth prior use the same π_p for a fair comparison. Even when π_p is not given, various methods can be used to estimate it [25, 55, 59].

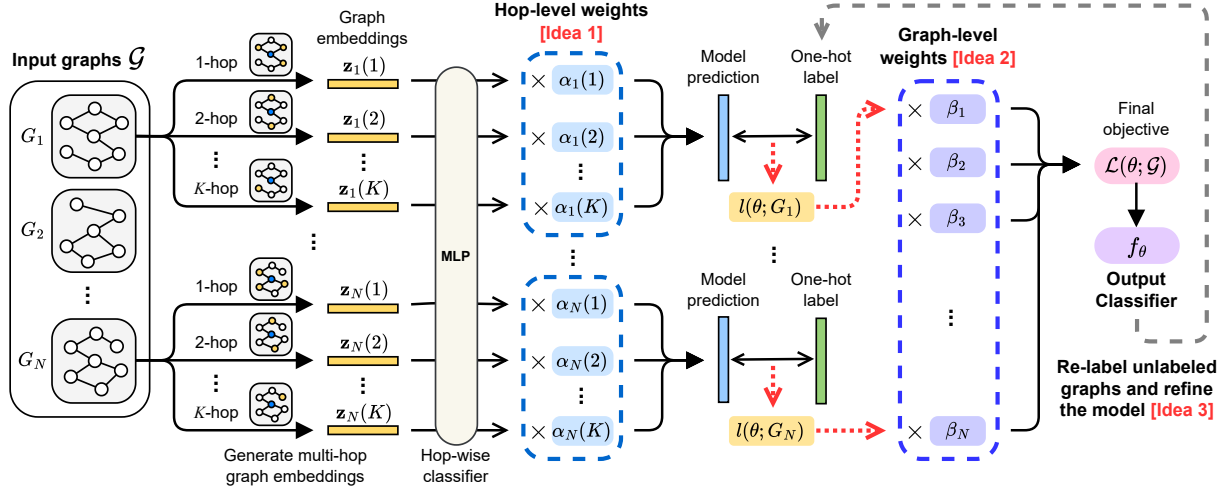


Figure 2: Overall structure of DELTA-PU. DELTA-PU generates multi-hop graph embeddings and applies hop-level weights α to emphasize informative ranges (idea 1). The instance-wise loss is further reweighted by graph-level weights β to downplay ambiguous unlabeled graphs (idea 2). DELTA-PU performs label refinement by iteratively updating pseudo-labels based on model predictions (idea 3).

2.3 Meta-Reweighting

Meta-reweighting [22, 58] aims to enhance the robustness of models by dynamically adjusting the importance of each training instance when clean labeled instances are scarce [14, 23]. The key idea is to assign higher weights to clean samples while suppressing the influence of noisy ones. This is achieved by aligning the gradient direction of each training instance with noise-free validation instances, enabling the model to focus on reliable supervision signals.

Let f_θ denote a model parameterized by θ and $l(\theta; x_i)$ be the loss incurred by instance x_i . Then the meta-reweighted loss is:

$$\mathcal{L}(\theta; \mathcal{D}_{\text{trn}}) = \sum_{x_i \in \mathcal{D}_{\text{trn}}} \epsilon_i \cdot l(\theta; x_i), \quad (1)$$

where ϵ_i denotes the importance weight for instance x_i and $\mathcal{D}_{\text{trn}}$ is a set of training instances. These weights are optimized as follows:

$$\epsilon_i \leftarrow \max \left(0, -\eta \frac{\partial}{\partial \epsilon_i} \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{x'_j \in \mathcal{D}_{\text{val}}} l(\theta(\epsilon); x'_j) \Big|_{\epsilon_i=0} \right), \quad (2)$$

where $\theta(\epsilon) = \theta - \eta \nabla_\theta \mathcal{L}(\theta; \mathcal{D}_{\text{trn}})$ with a learning rate η and $\mathcal{D}_{\text{val}}$ is the set of noise-free validation instances.

The gradient in Eq. (2) indicates how much the inclusion of training instance x_i affects the reduction of validation loss, and thus acts as a proxy for its contribution to generalization. Consequently, instances that lead to greater improvements on validation data receive higher weights during training.

While meta-reweighting has been effectively used in various domains such as noisy image classification [58, 62, 70], text classification [46], and domain adaptation [24], its application to graph domains remains underexplored. A recent study [20] applies meta-reweighting to node clustering, focusing solely on node-level tasks. Thus, it is not directly applicable to graph-level settings, where instance-wise ambiguity and structural reasoning are required.

To the best of our knowledge, DELTA-PU is the first PU graph classification method that leverages meta-reweighting. Its dual-level

reweighting mechanism effectively addresses both substructure ambiguity and instance unreliability—two key challenges inherent in PU graph classification.

3 Proposed Method

We describe our proposed DELTA-PU (see Figure 2 and Algorithm 1). The main challenges and our corresponding solutions are as follows.

- C1) Both positive and unlabeled graphs often include common motifs. How can we focus on discriminative substructures?** We assign hop-level importance scores on their task relevance, enabling the model to emphasize informative neighborhood ranges while suppressing less discriminative or redundant signals (Section 3.1).
- C2) Unlabeled instances contain mixed traits of both positive and negative graphs. How can we handle those ambiguous unlabeled graphs?** We assign graph-level importance scores for the unlabeled graphs to mitigate the impact of misleading samples throughout training (Section 3.2).
- C3) How can we further refine the model using the learned importance scores?** We re-label unlabeled instances based on their graph-level importance scores and perform one additional round of finetuning. This strategy allows the model to benefit from pseudo-supervised signals derived from reliable unlabeled graphs, leading to further performance improvements (Section 3.3).

3.1 Hop-level Reweighting

Discriminative signals in graph classification often stem from fine-grained substructures. However, identifying them is challenging under PU settings due to the absence of negative supervision. To address this, we propose hop-level reweighting, which learns the relative importance of each k -hop neighborhood.

We are given a set $\mathcal{G} = \mathcal{G}_{\text{pos}} \cup \mathcal{G}_{\text{unl}}$ of graphs, where $\mathcal{G}_{\text{pos}}$ and $\mathcal{G}_{\text{unl}}$ contain observed positive graphs and unlabeled ones, respectively. For every graph $G_i \in \mathcal{G}$, we first compute K hop-specific embeddings $\{z_i(1), z_i(2), \dots, z_i(K)\}$, where $z_i(k) \in \mathbb{R}^d$ is the graph representation of G_i derived from the k -hop neighborhood. Any graph encoder such as GIN [72] can be used for computing these embeddings.

To let the model focus on the most relevant receptive fields, we assign a learnable hop-level weight $\alpha_i(k)$ to the class probabilities computed based on each k -hop neighborhood. Let $f_\theta(\cdot)$ be a graph classifier with learnable parameters θ . The final prediction probability $f_\theta(G_i)$ for graph G_i is computed as a weighted sum of the hop-specific predictions:

$$f_\theta(G_i) = \sum_{k=1}^K \alpha_i(k) \text{softmax}(\text{lin}(z_i(k))) \quad (3)$$

where $\text{lin}(\cdot)$ is a linear layer that projects $z_i(k)$ to the same dimension as the number of classes.

Learning the hop-level weights α , however, is nontrivial in the absence of negative labels. Naively treating unlabeled graphs as negatives and using them as a signal for training α introduces substantial noise, leading to unreliable supervision.

To address this, we construct a clean validation set $\mathcal{G}_{\text{cln}}$ that provides a more reliable reference for learning the hop-level weights. $\mathcal{G}_{\text{cln}}$ consists of all observed positive graphs in $\mathcal{G}_{\text{pos}}$ and an equal number of pseudo-negative graphs $\tilde{\mathcal{G}}_{\text{neg}}$, i.e., $|\tilde{\mathcal{G}}_{\text{neg}}| = |\mathcal{G}_{\text{pos}}|$, which are selected from the unlabeled pool $\mathcal{G}_{\text{unl}}$. Specifically, we choose the unlabeled graphs with the lowest predicted probability of being positive under the current model. This pseudo-negative selection follows the PU ranking property [8]: lower-ranked unlabeled samples are more likely to be true negatives. Thus, keeping only a small bottom fraction makes it unlikely to include positives [87]. As $\tilde{\mathcal{G}}_{\text{neg}}$ depends on model predictions, $\mathcal{G}_{\text{cln}}$ is dynamically updated after each model update to reflect the current state of the model.

Given this clean validation set $\mathcal{G}_{\text{cln}}$, an intuition for computing α is to emphasize hop-level representations that contribute most to improving generalization on $\mathcal{G}_{\text{cln}}$. Specifically, for each graph G_i , we optimize the hop weights $\alpha_i = [\alpha_i(1), \alpha_i(2), \dots, \alpha_i(K)]$ for all K layers such that the gradients induced by the training loss align with those that reduce the validation loss on $\mathcal{G}_{\text{cln}}$, leveraging the meta-reweighting principle [58] (see Equation (2)). This encourages the model to emphasize receptive fields that contribute to generalization, rather than overfitting to noisy training signals.

Let $l(\theta; G_i)$ be a conventional classification loss (e.g., cross entropy) for graph G_i , with θ denoting the parameters of f_θ . Then each hop-level weight $\alpha_i(k)$ is computed as:

$$\alpha_i(k) \leftarrow \max \left(0, -\eta \frac{\partial}{\partial \alpha_i(k)} \frac{1}{|\mathcal{G}_{\text{cln}}|} \sum_{G'_j \in \mathcal{G}_{\text{cln}}} l(\theta(\alpha); G'_j) \Big|_{\alpha_i(k)=0} \right) \quad (4)$$

where $\theta(\alpha)$ denotes the model parameters after one gradient descent step with respect to α , i.e., $\theta(\alpha) = \theta - \xi \nabla_\theta \mathcal{L}(\theta; \mathcal{G})$. The overall objective function $\mathcal{L}(\theta; \mathcal{G})$ will be discussed in the following Section 3.2. To preserve the overall scale of graph embedding, we apply a softmax normalization:

$$\alpha_i(k) = [\alpha_i(1), \dots, \alpha_i(K)] \leftarrow \text{softmax}([\alpha_i(1), \dots, \alpha_i(K)] * T) \quad (5)$$

where T is the temperature parameter, set to 10 in experiments.

Algorithm 1: Overall process of DELTA-PU.

Input : A set $\mathcal{G} = \mathcal{G}_{\text{pos}} \cup \mathcal{G}_{\text{unl}}$ of graphs, where $\mathcal{G}_{\text{pos}}$ and $\mathcal{G}_{\text{unl}}$ denote the sets of positive and unlabeled graphs, respectively, threshold τ , and the aggregation depth K

Output : Best parameters θ of the graph classifier f_θ

/ Stage 1: Warm-up */*

- 1 $\tilde{y}_j \leftarrow 0, \forall G_j \in \mathcal{G}_{\text{unl}};$ // Assign initial pseudo-labels for $\mathcal{G}_{\text{unl}}$
- 2 Initialize f_θ by training without dual-level reweighting;

/ Stage 2: Training with Dual-level Reweighting */*

- 3 **for** $G_i \in \mathcal{G}$ **do**
- 4 **for** $k = 1, \dots, K$ **do**
- 5 $\alpha_i(k) \leftarrow 1/K;$ // Initialize hop-level weights
- 6 **end**
- 7 $\beta_i \leftarrow 1;$ // Initialize graph-level weights
- 8 **end**
- 9 **repeat**
- 10 Newly define a clean validation set $\mathcal{G}_{\text{cln}};$ // Graph set consisting of $\mathcal{G}_{\text{pos}}$ and pseudo-negatives from $\mathcal{G}_{\text{unl}}$
- 11 Update $\beta_j, \forall G_j \in \mathcal{G}_{\text{unl}};$ // Eq. (7)
- 12 Update $\alpha_i(k), \forall G_i \in \mathcal{G}, k = 1, \dots, K;$ // Eq. (4), (5)
- 13 **for** $G_i \in \mathcal{G}$ **do**
- 14 $l(\theta; G_i) = -\tilde{y}_i \log f_\theta(G_i) - (1 - \tilde{y}_i) \log(1 - f_\theta(G_i));$ // Eq. (3), (8)
- 15 **end**
- 16 $\mathcal{L}(\theta; \mathcal{G}) = |\mathcal{G}_{\text{pos}}|^{-1} \sum_{G_i \in \mathcal{G}_{\text{pos}}} \beta_i l(\theta; G_i) + |\mathcal{G}_{\text{unl}}|^{-1} \sum_{G_j \in \mathcal{G}_{\text{unl}}} \beta_j l(\theta; G_j);$ // Eq. (6)
- 17 $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta; \mathcal{G})$
- 18 **until** the maximum number of epochs is reached;

/ Stage 3: Fine-tuning */*

- 19 **for** $G_i \in \mathcal{G}_{\text{unl}}$ **do**
- 20 **if** $\beta_i < 0$ (Equation (7)) and $f_\theta(G_i) > \tau$ **then**
- 21 $\tilde{y}_i \leftarrow 1;$ // update pseudo-label of G_i
- 22 **end**
- 23 **end**
- 24 Fine-tune the classifier f_θ by running lines 9-18;

Our approach departs from existing meta-reweighting strategies in both its reweighting granularity and validation design. Unlike the original meta-reweighting method, which reweights individual training instances, we extend the idea to the structural level by modulating the contribution of each hop-specific representation. Moreover, whereas existing meta-reweighting-based methods rely on the assumption that a noise-free validation set is available—a condition that does not hold in PU settings due to the absence of confirmed negative labels—DELTA-PU dynamically constructs the clean validation set $\mathcal{G}_{\text{cln}}$ at each epoch. As a result, DELTA-PU progressively learns hop-level weights without requiring an explicitly given noise-free validation set.

3.2 Graph-level Reweighting

While hop-level reweighting highlights meaningful substructures, it alone cannot resolve the ambiguity in unlabeled graphs. Many of these graphs exhibit both positive and negative traits, and treating

them as fully reliable can introduce conflicting signals into the multi-hop aggregation, especially without confirmed negatives. As a result, the model may overfit to non-discriminative patterns.

To address this, we perform graph-level reweighting to suppress the influence of ambiguous unlabeled instances. Given the clean validation set $\mathcal{G}_{\text{cln}}$ (defined in Section 3.1), we assign an importance weight β_j to each graph $G_j \in \mathcal{G}$ such that the induced gradients align with those from $\mathcal{G}_{\text{cln}}$. This alignment encourages the model to prioritize graphs that support the same decision boundary as $\mathcal{G}_{\text{cln}}$.

The reweighted objective function $\mathcal{L}(\theta; \mathcal{G})$ of DELTA-PU is expressed as follows:

$$\mathcal{L}(\theta; \mathcal{G}) = \frac{1}{|\mathcal{G}_{\text{pos}}|} \sum_{G_i \in \mathcal{G}_{\text{pos}}} \beta_i l(\theta; G_i) + \frac{1}{|\mathcal{G}_{\text{unl}}|} \sum_{G_j \in \mathcal{G}_{\text{unl}}} \beta_j l(\theta; G_j) \quad (6)$$

where $l(\theta; G_i)$ is the classification loss for graph G_i . The classifier f_θ used for computing $l(\theta; G_i)$ aggregates hop-specific predictions using the weights $\alpha_i(k)$, as defined in Equation (3). Thus, both hop-level weights α and graph-level weights β jointly influence the final loss $\mathcal{L}$ through prediction and instance reweighting, respectively.

Similar to the process of updating α in Equation (4), we update the graph-level weight β_j under the supervision of $\mathcal{G}_{\text{cln}}$:

$$\beta_j \leftarrow \max \left(0, -\eta \frac{\partial}{\partial \beta_j} \frac{1}{|\mathcal{G}_{\text{cln}}|} \sum_{G'_i \in \mathcal{G}_{\text{cln}}} l(\theta(\beta); G'_i) \Big|_{\beta_j=0} \right) \quad (7)$$

where $\theta(\beta)$ is the updated model parameter after a gradient descent step with learning rate ξ , i.e., $\theta(\beta) = \theta - \xi \nabla_\theta \mathcal{L}(\theta; \mathcal{G})$.

To ensure consistent scaling, we first normalize all graph weights by dividing them by their maximum value, aligning their scale to the $[0, 1]$ range. We then set the weights of observed positive instances to 1, as they provide the most reliable supervision signal in the absence of confirmed negatives. For unlabeled instances, we apply a rescaling step with a soft lower bound of $1 - \pi_p$, where $1 - \pi_p$ is the expected probability that an unlabeled instance is negative. This lower bound prevents excessive underweighting and reflects the implicit assumption that unlabeled examples are more likely to be negative. This is particularly important in graph classification tasks, where the number of training instances is limited, and underweighting too many potentially informative examples could lead to unstable or biased gradient updates.

3.3 Three-stage Learning

To robustly train the model under PU setting, we employ a three-stage learning framework that progressively enhances reliability in both instance selection and label quality. The full procedure is summarized in Algorithm 1.

In the first stage (warm-up stage), all unlabeled instances are heuristically assumed to be negative. The model is trained using standard supervised learning without applying hop- or graph-level importance weights (α and β), as shown in lines 1-2 of Algorithm 1. This stage serves to establish a stable initialization. The optimizer is re-initialized before proceeding to the next phase.

In the second stage (reweighting stage), we incorporate the hop- and graph-level weights into training. These weights are learned in conjunction with model parameters to emphasize informative substructures and downweight unreliable unlabeled instances (lines

9-18 of Algorithm 1). The optimizer is re-initialized before entering the next stage.

In the final stage (fine-tuning stage), a subset of graphs in $\mathcal{G}_j \in \mathcal{G}_{\text{unl}}$, which were initially treated as negatives, is selected and relabeled based on the learned dual-level weights and model predictions. We then fine-tune the model by jointly updating the model parameters and dual-level weights, as in the second stage. An unlabeled graph G_i is relabeled with $\tilde{y}_i = 1$ if two conditions are met: a) its graph-level weight β_i in Equation (7) is smaller than 0, indicating that treating G_i as negative degrades validation performance, and b) the trained model predicts it as positive with high confidence, i.e., $f_\theta(G_i) > \tau$, where $\tau \in (0, 1)$ is a fixed threshold set to 0.95 in our experiments. These conditions suggest that the initial assumption of negativity was likely incorrect, and the instance should instead be treated as positive. This relabeling and fine-tuning process corresponds to lines 19-24 in Algorithm 1.

Our three-stage learning framework is agnostic to the choice of loss function; any instance-wise loss can be employed. In our implementation, we adopt the binary cross-entropy loss between the model prediction $f_\theta(G_i)$ and the supervision label $\tilde{y}_i$, which is defined as follows:

$$l(\theta; G_i) = -\tilde{y}_i \log f_\theta(G_i) - (1 - \tilde{y}_i) \log(1 - f_\theta(G_i)). \quad (8)$$

This loss is used consistently across all stages of training. The label $\tilde{y}_i$ is set to 1 for observed positives and for relabeled unlabeled instances in the fine-tuning stage, and 0 otherwise.

3.4 Time Complexity

DELTA-PU is scalable to large datasets as its overall training time scales linearly with the total number of nodes and edges across multiple input graphs. For simplicity, we assume a GIN classifier with K GIN layers, each consisting of an aggregation step followed by an MLP with n_m fully connected layers and d hidden units.

LEMMA 1. *We are given a set $\mathcal{G} = \mathcal{G}_{\text{pos}} \cup \mathcal{G}_{\text{unl}}$ of graphs where $\mathcal{G}_{\text{pos}}$ is the set of observed positive graphs and $\mathcal{G}_{\text{unl}}$ is the set of unlabeled graphs. Then the overall time complexity of training DELTA-PU is*

$$O(n_{\text{ep}} K^2 d (E + V n_m d))$$

where n_{ep} is the number of epochs, E is the total number of edges, and V is the total number of nodes across all graphs in $\mathcal{G}$.

PROOF. Each epoch of DELTA-PU consists of three major steps: a) updating model parameters θ of the classifier f , b) updating hop-level weights $\alpha_i(k)$ for all graphs $G_i \in \mathcal{G}$ and GIN layers $k = 1, \dots, K$, and c) updating graph-level weights β_j for all graphs $G_j \in \mathcal{G}$.

First, updating the parameters of f over $\mathcal{G}$ for each epoch yields a total time complexity of

$$O(Kd \sum_{G_i \in \mathcal{G}} (|\mathcal{E}_i| + |\mathcal{V}_i| n_m d)) \quad (9)$$

since each GIN layer performs one aggregation step followed by n_m linear transformations with hidden dimension d . As $E = \sum_{G_i \in \mathcal{G}} |\mathcal{E}_i|$ and $V = \sum_{G_i \in \mathcal{G}} |\mathcal{V}_i|$, Equation (9) simplifies to

$$O(Kd (E + V n_m d)). \quad (10)$$

Second, DELTA-PU computes hop-level weights $\alpha_i(k)$ for each GIN layer k of every training graph $G_i \in \mathcal{G}$ using a meta-reweighting scheme (Equation (4)). Computing $\frac{1}{|\mathcal{G}_{\text{cln}}|} \sum_{G'_i \in \mathcal{G}_{\text{cln}}} l(\theta(\alpha); G'_i)$ over

the clean set $\mathcal{G}_{\text{cln}}$ requires $\mathcal{O}\left(Kd \sum_{G_i' \in \mathcal{G}_{\text{cln}}} (|\mathcal{E}_i'| + |\mathcal{V}_i'|n_{\text{md}})\right)$ where $\mathcal{E}_i'$ and $\mathcal{V}_i'$ denote the sets of edges and nodes of $\mathcal{G}_{\text{cln}}$, respectively. Thus, computing $\alpha_i(k)$ for all K layers and all graphs $\mathcal{G}$ takes

$$\mathcal{O}\left(K^2d \sum_{G_i \in \mathcal{G}} (|\mathcal{E}_i| + |\mathcal{V}_i|n_{\text{md}})\right), \quad (11)$$

as each $\alpha_i(k)$ involves backpropagation through both training graphs $\mathcal{G}$ and the clean set $\mathcal{G}_{\text{cln}}$, which is a subset of $\mathcal{G}$. Then Equation (11) simplifies to

$$\mathcal{O}\left(K^2d (E + Vn_{\text{md}})\right), \quad (12)$$

since $E = \sum_{G_i \in \mathcal{G}} |\mathcal{E}_i|$ and $V = \sum_{G_i \in \mathcal{G}} |\mathcal{V}_i|$.

Similarly, the time complexity of updating graph-level weights β_j for all graphs $G_j \in \mathcal{G}$ is

$$\mathcal{O}(Kd (E + Vn_{\text{md}})). \quad (13)$$

since it takes $\mathcal{O}(Kd (E_j + V_jn_{\text{md}}))$ to compute each β_j .

With Equations (10), (12), (13), the total time complexity of training DELTA-PU for n_{ep} epochs is

$$\mathcal{O}\left(n_{\text{ep}}K^2d(E + Vn_{\text{md}})\right),$$

which is linear in the number of total nodes and edges in $\mathcal{G}$. $\square$

4 Experiments

We perform experiments to answer the following questions:

- Q1) Classification performance.** Does the proposed DELTA-PU outperform existing PU graph classification methods across various benchmarks? (Section 4.2).
- Q2) Effect of three-stage learning.** How does the three-stage learning framework affect classification performance under PU supervision? (Section 4.3).
- Q3) Effect of dual-level reweighting.** How do the hop- and graph-level importance weights individually and jointly affect the performance of DELTA-PU? (Section 4.4).
- Q4) Scalability analysis.** Does DELTA-PU show linear time complexity with the graph size? (Section 4.5).

4.1 Experimental Setup

We introduce the experimental setup including datasets, baseline methods, and other training details.

4.1.1 Datasets. We use seven benchmark datasets, whose statistics are summarized in Table 5 (Appendix A.3). MUTAG, NCI1, NCI109, and PROTEINS [45] consist of molecular graphs, where each graph represents a chemical compound and the classification task involves predicting a property or function of the molecule—such as toxicity or enzyme activity. Facebook dataset [45] consists of temporal graphs from the New Orleans Facebook community, used to classify whether vertex labels arise from a dissemination process or not. BA2Motif [43] and BAMultiShape [2] are synthetic datasets widely used in graph classification area.

To simulate the PU setting, we follow the standard protocol [25] by masking a portion of the positive labels. Specifically, given the original dataset with binary labels, we randomly select a subset of positive graphs to retain as observed positives, while treating the remaining positive and all negative graphs as unlabeled. We perform experiments with three settings for the observed positive ratio:

$r_{\text{obs}} \in \{0.1, 0.2, 0.3\}$. For the ENZYMES dataset, which contains six classes, we convert it to a binary classification task by assigning the most frequent class as positive and the rest as negative.

4.1.2 Baselines. We compare DELTA-PU with recent PU learning methods and graph readout methods. To ensure a fair comparison, all models including DELTA-PU employ the GIN [72] architecture as the graph encoder, which has demonstrated strong performance across various graph classification tasks. Following the setup in [81], we use a 5-layer GIN with a hidden dimension of 64.

For PU methods, we consider WCE (weighted cross-entropy), URE [7], NNRE [30], and Dist-PU [18, 84]. WCE uses balanced CE loss by averaging the mean losses of positive and unlabeled (treated as negative) instances for graph classifier training. URE is an objective function specifically designed for PU learning to estimate the unknown negative risk. NNRE forces the loss to be positive and increases the stability of learning. Dist-PU aligns predicted and true label distributions based on the known class prior.

For graph readout methods, we consider MeanPool, MaxPool, AttenPool [38], and GMTPool [3]. All the readout methods operate on the node representations obtained from the GIN encoder. Mean and Max pooling perform global aggregation by computing the average and maximum over node features, respectively. AttenPool (global attention pooling) leverages a gated neural network to assign attention scores to node embeddings, enabling the model to highlight informative regions of the graph. GMTPool (graph multi-set transformer) employs a transformer-based readout architecture that learns adaptive multiset representations, providing expressive and permutation-invariant graph-level embeddings. To adapt these baselines to the PU setting, we replace their original loss with NNRE, which consistently improves performance. Further details on the implementation of each baseline are provided in Appendix A.1.

4.1.3 Training details and evaluation metrics. Following the experimental process of GIN [72], we run 10-fold cross-validation for evaluation and report the mean performance across the 10 test folds. We use Adam optimizer with an initial learning rate of 0.01, which is halved every 50 epochs following [81]. Each baseline is trained for 100 epochs, as in previous work [27]. We train DELTA-PU using a three-stage procedure where each stage is run for 100 epochs. We use macro F1-score as the primary evaluation metric, following common practice in PU learning [44, 79]. For completeness, we also report accuracy.

4.2 Classification Performance

We compare the PU graph classification performance of DELTA-PU and various baselines in Table 1 under three observed positive ratios ($r_{\text{obs}} \in \{0.1, 0.2, 0.3\}$). We also report additional results on BA2Motif in Appendix A.2. The Avg. Rank columns are obtained by ranking all methods on each dataset in terms of F1 and accuracy, and then averaging these ranks across datasets.

DELTA-PU consistently achieves the highest macro-F1, which is a more reliable metric than accuracy under class imbalance in PU settings. DELTA-PU also ranks first in accuracy on the majority of datasets, and achieves the lowest (i.e., best) average rank in both F1 and accuracy across all observed ratios. In particular, DELTA-PU demonstrates strong performance on larger datasets such as

Table 1: Classification performance of DELTA-PU and baselines in terms of macro-F1 and accuracy under three observed positive ratios ($r_{\text{obs}} \in \{0.1, 0.2, 0.3\}$). Due to the inherent class imbalance in PU learning, F1-score serves as a more informative metric than accuracy in assessing the overall model reliability. The best and second-best performances are highlighted in bold and underlined, respectively. Note that DELTA-PU is consistently ranked first or second across most datasets.

Observed positive ratio $r_{\text{obs}} = 0.1$													
Model	MUTAG		NCI1		NCI109		PROTEINS		BAMultiShape		Facebook		Avg. Rank
	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	
AddPool	59.20 (6)	66.52 (7)	49.59 (5)	52.87 (6)	52.27 (3)	54.13 (6)	<u>54.14 (2)</u>	62.73 (1)	68.03 (5)	70.60 (4)	<u>77.68 (2)</u>	78.10 (2)	3.83 4.33
MaxPool	63.26 (3)	71.35 (4)	<u>53.61 (2)</u>	<u>56.11 (2)</u>	<u>53.75 (2)</u>	<u>54.88 (2)</u>	46.31 (6)	61.37 (4)	65.00 (6)	67.30 (7)	76.10 (3)	76.88 (3)	3.67 3.67
AttenPool	60.72 (5)	<u>72.31 (2)</u>	51.29 (3)	54.38 (4)	51.41 (4)	54.66 (3)	49.15 (3)	61.37 (5)	<u>72.54 (2)</u>	<u>73.80 (2)</u>	79.05 (1)	79.71 (1)	3.00 2.83
GMTPool	60.96 (4)	70.20 (5)	33.31 (9)	49.95 (9)	33.17 (9)	49.63 (9)	40.86 (9)	60.29 (7)	61.40 (7)	68.90 (6)	75.21 (5)	76.07 (5)	7.17 6.83
WCE	40.97 (8)	45.09 (9)	47.41 (6)	52.87 (7)	49.18 (6)	54.23 (4)	41.27 (8)	60.38 (6)	49.37 (9)	57.30 (9)	58.53 (8)	62.63 (8)	7.50 7.17
URE	41.48 (7)	67.02 (6)	43.48 (7)	51.21 (8)	45.21 (7)	51.66 (8)	47.29 (5)	57.68 (9)	51.07 (8)	61.20 (8)	73.44 (7)	74.27 (7)	6.83 7.67
NNRE	<u>65.34 (2)</u>	73.98 (1)	50.36 (4)	54.89 (3)	51.00 (5)	54.20 (5)	49.15 (4)	61.73 (3)	68.40 (4)	70.30 (5)	75.30 (4)	76.50 (4)	3.83 3.50
Dist-PU	39.92 (9)	66.49 (8)	43.08 (8)	52.89 (5)	38.21 (8)	51.73 (7)	41.47 (7)	59.39 (8)	70.29 (3)	72.30 (3)	55.72 (9)	61.92 (9)	7.33 6.67
DELTA-PU (ours)	69.75 (1)	71.40 (3)	61.95 (1)	62.46 (1)	62.43 (1)	62.66 (1)	57.34 (1)	<u>62.27 (2)</u>	83.08 (1)	83.20 (1)	74.58 (6)	75.67 (6)	1.83 2.33

Observed positive ratio $r_{\text{obs}} = 0.2$													
Model	MUTAG		NCI1		NCI109		PROTEINS		BAMultiShape		Facebook		Avg. Rank
	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	
AddPool	64.16 (3)	71.84 (4)	49.77 (3)	54.79 (4)	49.84 (4)	54.28 (4)	46.17 (5)	59.83 (6)	69.18 (5)	71.30 (5)	78.43 (3)	79.29 (4)	3.83 4.50
MaxPool	62.72 (4)	72.25 (3)	51.54 (2)	55.25 (2)	54.94 (2)	56.21 (2)	50.80 (2)	61.90 (2)	74.48 (3)	75.80 (3)	76.78 (5)	77.88 (5)	3.00 2.83
AttenPool	60.09 (5)	<u>72.34 (2)</u>	48.70 (5)	54.35 (5)	51.44 (3)	55.32 (3)	50.55 (3)	61.82 (3)	67.96 (6)	70.40 (6)	78.29 (4)	79.39 (3)	4.33 3.67
GMTPool	49.62 (6)	63.22 (8)	33.31 (9)	49.95 (9)	33.17 (9)	49.63 (9)	38.46 (9)	59.75 (8)	63.29 (8)	70.20 (7)	67.82 (8)	70.87 (8)	8.17 8.17
WCE	48.00 (7)	49.88 (9)	48.66 (6)	54.04 (7)	45.15 (7)	52.66 (5)	38.52 (8)	59.39 (9)	64.34 (7)	67.20 (8)	69.26 (7)	71.16 (7)	7.00 7.50
URE	43.34 (8)	68.07 (6)	47.46 (7)	53.23 (8)	46.71 (6)	52.36 (6)	44.11 (6)	59.75 (7)	53.77 (9)	62.70 (9)	71.17 (6)	73.30 (6)	7.00 7.00
NNRE	<u>66.34 (2)</u>	75.53 (1)	49.25 (4)	54.89 (3)	47.68 (5)	52.03 (7)	49.18 (4)	60.83 (4)	70.05 (4)	71.60 (4)	<u>79.38 (2)</u>	<u>79.89 (2)</u>	3.50 3.50
Dist-PU	39.92 (9)	66.49 (7)	46.14 (8)	54.23 (6)	35.63 (8)	50.64 (8)	41.81 (7)	59.93 (5)	<u>75.77 (2)</u>	<u>77.70 (2)</u>	57.08 (9)	62.70 (9)	7.17 6.17
DELTA-PU (ours)	67.87 (1)	71.32 (5)	62.91 (1)	63.60 (1)	62.93 (1)	63.70 (1)	61.35 (1)	64.15 (1)	88.68 (1)	88.70 (1)	87.49 (1)	87.53 (1)	1.00 1.67

Observed positive ratio $r_{\text{obs}} = 0.3$													
Model	MUTAG		NCI1		NCI109		PROTEINS		BAMultiShape		Facebook		Avg. Rank
	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	
AddPool	58.22 (5)	66.87 (7)	<u>55.54 (2)</u>	<u>58.74 (2)</u>	50.88 (3)	54.57 (5)	44.89 (5)	59.48 (8)	63.29 (8)	67.90 (7)	76.99 (3)	78.00 (5)	4.33 5.67
MaxPool	64.03 (4)	72.34 (4)	55.30 (3)	58.20 (3)	<u>54.05 (2)</u>	<u>56.36 (2)</u>	<u>47.99 (2)</u>	61.72 (3)	<u>75.96 (2)</u>	<u>77.00 (2)</u>	76.59 (5)	<u>78.50 (2)</u>	3.00 2.67
AttenPool	65.36 (3)	<u>75.00 (2)</u>	55.21 (4)	58.00 (4)	49.97 (4)	55.08 (3)	45.49 (4)	60.02 (4)	<u>67.20 (6)</u>	69.80 (6)	76.72 (4)	78.20 (4)	4.17 3.83
GMTPool	53.83 (7)	70.18 (5)	33.31 (9)	49.95 (9)	33.17 (9)	49.63 (9)	38.47 (9)	59.93 (6)	71.21 (4)	75.90 (3)	67.44 (8)	71.96 (7)	7.67 6.50
WCE	56.55 (6)	56.96 (9)	51.73 (6)	55.74 (6)	48.59 (5)	54.74 (4)	40.04 (8)	59.66 (7)	64.20 (7)	67.70 (8)	<u>77.11 (2)</u>	78.27 (3)	5.67 6.17
URE	42.41 (8)	67.54 (6)	41.81 (7)	52.53 (8)	42.27 (7)	51.06 (7)	43.37 (6)	59.02 (9)	51.84 (9)	61.60 (9)	68.17 (7)	70.79 (8)	7.33 7.83
NNRE	<u>65.41 (2)</u>	74.94 (3)	51.85 (5)	56.37 (5)	46.39 (6)	54.23 (6)	47.34 (3)	<u>61.81 (2)</u>	75.04 (3)	75.80 (4)	76.35 (6)	77.69 (6)	4.17 4.33
Dist-PU	39.92 (9)	66.49 (8)	40.77 (8)	53.06 (7)	34.63 (8)	50.55 (8)	41.99 (7)	60.02 (5)	69.51 (5)	72.70 (5)	63.48 (9)	67.12 (9)	7.67 7.00
DELTA-PU (ours)	71.44 (1)	77.11 (1)	64.62 (1)	65.50 (1)	65.91 (1)	66.47 (1)	61.41 (1)	64.15 (1)	90.89 (1)	90.90 (1)	87.70 (1)	87.74 (1)	1.00 1.00

NCI1, NCI109, and PROTEINS. Such consistent and scalable performance highlights the robustness of DELTA-PU across diverse graph structures and levels of label imbalance.

4.3 Effect of Three-stage Learning

We evaluate how the macro-F1 of DELTA-PU evolves throughout the three-stage learning procedure (warm-up, meta-reweighting, and fine-tuning) in Figure 3.

In the first stage (warm-up stage), the model is trained by treating all unlabeled graphs as negatives without applying hop- or graph-level importance weights. Performance remains relatively flat during this phase, serving primarily to provide stable initialization while avoiding the early influence of noisy supervision.

In the second stage (reweighting stage), both hop-level and graph-level weights are incorporated, which leads to a substantial performance boost. The model learns to focus on informative substructures and down-weights harmful instances, resulting in sharp

improvements in F1-score. Notably, this phase shows the effectiveness of dual-level reweighting in distinguishing valuable training signals from noise.

In the final stage (fine-tuning stage), pseudo-labels for certain unlabeled instances are revised based on the learned importance scores and the model’s prediction confidence. Training with these relabeled instances yields further performance gains, as observed in the final F1-score trajectory.

4.4 Effect of Dual-level Reweighting

We conduct an ablation study to evaluate the impact of hop-level (α) and graph-level (β) importance weights in DELTA-PU. DELTA-PU-w/o- α and DELTA-PU-w/o- β denote variants of DELTA-PU without hop-level and graph-level importance weighting, respectively, while DELTA-PU-w/o- α & β removes both.

Note that each component plays a crucial role in performance: removing either hop-level (α) or graph-level (β) weights degrades

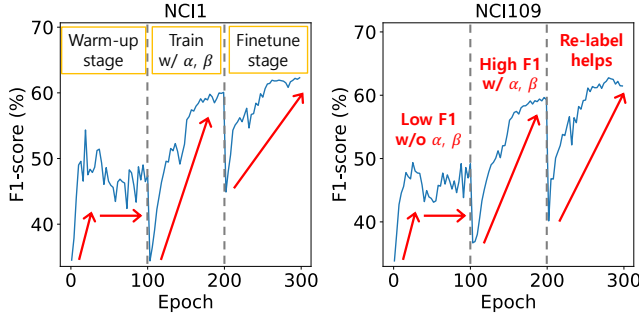


Figure 3: F1-score progression over three training stages in NCI1 and NCI109 datasets. The warm-up stage trains the model without graph- or hop-level importance weights (α and β). In the training stage, both levels of weights are learned, resulting in a notable performance improvement. The finetuning stage further improves F1 by training on re-labeled instances based on the learned dual-level weights.

Table 2: Ablation study on the contribution of hop-level (α) and graph-level (β) importance weights in DELTA-PU to performance. The removal of either component leads to a consistent drop in both F1 and accuracy, showing the complementary contribution of both modules.

Dataset	Metric	w/o α	w/o β	w/o $\alpha\&\beta$	DELTA-PU
MUTAG	F1	74.11 (1)	66.08 (4)	67.56 (3)	67.87 (2)
	Acc	76.70 (1)	67.49 (4)	68.66 (3)	71.32 (2)
NCI1	F1	62.86 (2)	60.95 (3)	59.60 (4)	62.91 (1)
	Acc	63.70 (1)	62.55 (3)	61.51 (4)	63.60 (2)
NCI109	F1	61.91 (2)	57.94 (4)	57.96 (3)	62.93 (1)
	Acc	62.32 (2)	60.60 (4)	60.77 (3)	63.70 (1)
PROTEINS	F1	59.75 (2)	56.06 (3)	54.38 (4)	61.35 (1)
	Acc	62.63 (4)	63.97 (3)	64.07 (2)	64.15 (1)
BAMultiShape	F1	86.73 (2)	78.52 (3)	75.38 (4)	88.68 (1)
	Acc	86.80 (2)	79.00 (3)	76.00 (4)	88.70 (1)
Facebook	F1	84.52 (2)	82.24 (4)	82.98 (3)	87.49 (1)
	Acc	84.61 (2)	82.80 (4)	83.32 (3)	87.53 (1)

the results. These findings confirm that dual-level reweighting is complementary, and that both components jointly contribute to the overall effectiveness of DELTA-PU.

4.5 Scalability Analysis

To assess the computational scalability of DELTA-PU, we measure the time cost per epoch under varying graph and dataset sizes. Figure 4 reports the results of two complementary experiments. In the first setting (Figure 4, left), we control the size of each graph by randomly dropping edges. For each dataset, we vary the edge retention ratio from 0.1 to 0.9. In the second setting (Figure 4, right), we reduce the dataset size by randomly subsampling graphs, which naturally reduces the total number of nodes and edges. As in the first setting, we vary the graph retention ratio from 0.1 to 0.9.

In both experiments, we observe that the training time per epoch increases linearly with the number of edges or graphs. This indicates

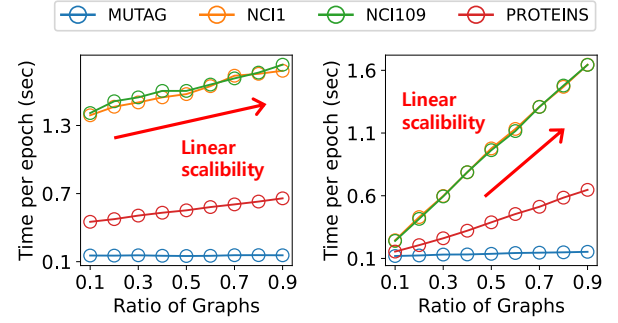


Figure 4: Running time per epoch as the graph dataset size is controlled in two different ways. First, we retain a fraction of edges in each graph from 0.1 to 0.9 (left). Second, we subsample the dataset by randomly selecting a fraction of graphs (right). In both settings, DELTA-PU exhibits linear scalability across all datasets. This shows the practicality and low-overhead of DELTA-PU for large-scale graphs.

that DELTA-PU scales efficiently with respect to both graph size and dataset size.

5 Conclusion

In this paper, we propose DELTA-PU for accurate positive-unlabeled (PU) graph classification. DELTA-PU addresses two key challenges in PU graph classification: shared substructures across classes that obscure fine-grained discriminative patterns, and ambiguous unlabeled graphs that can mislead the model in the absence of confirmed negatives. To address these, DELTA-PU performs dual-level meta-reweighting by prioritizing informative hop-level receptive fields during pooling and downweighting unreliable unlabeled graphs, followed by a fine-tuning stage that revises pseudo-labels using model predictions and graph-level weights. Experimental results on many real-world datasets show that DELTA-PU consistently outperforms existing PU graph classification methods.

GenAI Usage

OpenAI ChatGPT was used solely for post-editing (typos and minor grammatical corrections). It was not involved in conceptual development, data analysis, or content generation. The authors bear full responsibility for the manuscript’s intellectual content.

Acknowledgments

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) [No.2022-0-00641, XVoice: Multi-Modal Voice Meta Learning], [No.RS-2025-25442338, AI star Fellowship Support Program (Seoul National University)], [No.RS-2024-00509257, Global AI Frontier Lab], and [No.RS-2021-II211343, Artificial Intelligence Graduate School Program (Seoul National University)]. The Institute of Engineering Research at Seoul National University and the Institute of Computer Technology at Seoul National University provided research facilities for this work. U Kang is the corresponding author.

References

- [1] David Ahméd-Aristizabal, Mohammad Ali Armin, Simon Denman, Clinton Fookes, and Lars Petersson. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors* (2021).
- [2] Steve Azzolin, Antonio Longa, Pietro Barbiero, Pietro Liò, and Andrea Passerini. 2023. Global explainability of gnns via logic combination of learned concepts. *ICLR* (2023).
- [3] Jinheon Baek, Minki Kang, and Sung Ju Hwang. 2021. Accurate learning of graph representations with graph multiset pooling. *ICLR* (2021).
- [4] Na Le Dang, Tyler B Hughes, Grover P Miller, and S Joshua Swamidass. 2017. Computational approach to structural alerts: furans, phenols, nitroaromatics, and thiophenes. *Chemical research in toxicology* (2017).
- [5] Yingdong Dou, Kai Shu, Congying Xia, Philip S Yu, and Lichao Sun. 2021. User preference-aware fake news detection. In *SIGIR*.
- [6] Hongwei Du, Jiamin Wang, Jian Hui, Lanting Zhang, and Hong Wang. 2024. DenseGNN: universal and scalable deeper graph neural networks for high-performance property prediction in crystals and molecules. *npj Computational Materials* (2024).
- [7] Marthinus Christoffel du Plessis, Gang Niu, and Masashi Sugiyama. 2014. Analysis of Learning from Positive and Unlabeled Data. In *NeurIPS*.
- [8] Charles Elkan and Keith Noto. 2008. Learning classifiers from only positive and unlabeled data. In *KDD*.
- [9] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [10] Hongyang Gao and Shuiwang Ji. 2019. Graph u-nets. In *ICML*.
- [11] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *NeurIPS* (2017).
- [12] Yu-Guan Hsieh, Gang Niu, and Masashi Sugiyama. 2019. Classification from Positive, Unlabeled and Biased Negative Data. In *ICML*.
- [13] Jingjia Huang, Zhangheng Li, Nannan Li, Shan Liu, and Ge Li. 2019. AttPool: Towards Hierarchical Feature Representation in Graph Convolutional Networks via Attention Mechanism. In *ICCV*.
- [14] Ka Hyun Park, Junghun Kim, and U Kang. 2025. Accurate semi-supervised automatic speech recognition for ordinary and characterized speeches via multi-hypotheses-based curriculum learning. *PLoS One* (2025).
- [15] Jun-Gi Jang and U Kang. 2022. Dpar2: Fast and scalable parafac2 decomposition for irregular dense tensors. In *ICDE*.
- [16] Jun-Gi Jang, Jeongyoung Lee, Jiwon Park, and U Kang. 2022. Accurate PARAFAC2 decomposition for temporal irregular tensors with missing values. In *Big Data*.
- [17] Jun-Gi Jang, Jeongyoung Lee, Yong-chan Park, and U Kang. 2023. Fast and accurate dual-way streaming parafac2 for irregular tensors-algorithm and application. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.
- [18] Yangbangan Jiang, Qianqian Xu, Yunrui Zhao, Zhiyong Yang, Peisong Wen, Xiaochun Cao, and Qingming Huang. 2023. Positive-unlabeled learning with label distribution alignment. *TPAMI* (2023).
- [19] Woojeong Jin, Jinhong Jung, and U Kang. 2019. Supervised and extended restart in random walks for ranking and link prediction in networks. *PLoS one* (2019).
- [20] Hyeonsoo Jo, Fanchen Bu, and Kijung Shin. 2023. Robust Graph Clustering via Meta Weighting for Noisy Graphs. In *CIKM*.
- [21] Junghun Kim, Jinhong Jung, and U Kang. 2021. Compressing deep graph convolution network with multi-staged knowledge distillation. *Plos one* (2021).
- [22] Junghun Kim, Ka Hyun Park, Jun-Gi Jang, and U Kang. 2024. Fast and accurate domain adaptation for irregular tensor decomposition. In *KDD*.
- [23] Junghun Kim, Ka Hyun Park, and U Kang. 2024. Accurate Semi-supervised Automatic Speech Recognition via Multi-hypotheses-Based Curriculum Learning. In *PAKDD*.
- [24] Junghun Kim, Ka Hyun Park, Hoyoung Yoon, and U Kang. 2024. Domain-Aware Data Selection for Speech Classification via Meta-Reweighting. In *Interspeech*.
- [25] Junghun Kim, Ka Hyun Park, Hoyoung Yoon, and U Kang. 2025. Accurate Graph-based Multi-Positive Unlabeled Learning via Disentangled Multi-view Feature Propagation. In *KDD*.
- [26] Junghun Kim, Ka Hyun Park, Hoyoung Yoon, and U Kang. 2025. Accurate Link Prediction for Edge-Incomplete Graphs via PU Learning. In *AAAI*.
- [27] Minjun Kim, Jaehyeon Choi, SeungJoo Lee, Jinhong Jung, and U Kang. 2025. AugWard: Augmentation-Aware Representation Learning for Accurate Graph Classification. *PAKDD* (2025).
- [28] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [29] Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. *ICLR* (2017).
- [30] Ryuichi Kiryo, Gang Niu, Marthinus Christoffel du Plessis, and Masashi Sugiyama. 2017. Positive-Unlabeled Learning with Non-Negative Risk Estimator. In *NeurIPS*.
- [31] Junhyun Lee, Inyeop Lee, and Jaewoo Kang. 2019. Self-attention graph pooling. In *International conference on machine learning*.
- [32] SeungJoo Lee, Yong-Chan Park, and U Kang. 2024. Accurate Coupled Tensor Factorization with Knowledge Graph. In *(BigData)*.
- [33] Fabien Letouzey, François Denis, and Régis Gilleron. 2000. Learning from positive and unlabeled examples. In *ALT*.
- [34] Michelle M Li, Kexin Huang, and Marinka Zitnik. 2022. Graph representation learning in biomedicine and healthcare. *Nature Biomedical Engineering* (2022).
- [35] Wen-Sheng Li, Bing Lee, and Kwok-Shing Leung. 2009. Learning from positive and unlabeled examples using maximum margin clustering. *IJDWM* (2009).
- [36] Xianxian Li, Zeming Gan, Qiyu Li, Bin Qu, Jinyan Wang, et al. 2025. Rethinking the impact of noisy labels in graph classification: A utility and privacy perspective. *Neural Networks* (2025).
- [37] Xiaoli Li and Bing Liu. 2003. Learning to classify texts using positive and unlabeled data. In *IJCAI*.
- [38] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2016. Gated graph sequence neural networks. *ICLR* (2016).
- [39] Bing Liu, Wee Sun Lee, Philip S Yu, and Xiaoli Li. 2002. Partially supervised classification of text documents. In *ICML*.
- [40] Chuang Liu, Yibing Zhan, Jia Wu, Chang Li, Bo Du, Wenbin Hu, Tongliang Liu, and Dacheng Tao. 2023. Graph Pooling for Graph Neural Networks: Progress, Challenges, and Opportunities. In *IJCAI*.
- [41] Sheng Liu, Jonathan Niles-Weed, Narges Razavian, and Carlos Fernandez-Granda. 2020. Early-Learning Regularization Prevents Memorization of Noisy Labels. In *NeurIPS*.
- [42] Yonghao Liu, Lan Huang, Bowen Cao, Ximing Li, Fausto Giunchiglia, Xiaoyue Feng, and Renchu Guan. 2024. A simple but effective approach for unsupervised few-shot graph classification. In *WWW*.
- [43] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized explainer for graph neural network. *NeurIPS* (2020).
- [44] Shuangxun Ma and Ruisheng Zhang. 2017. PU-LP: A novel approach for positive and unlabeled learning by label propagation. In *International Conference on Multimedia and Expo*.
- [45] Christopher Morris, Nils M Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TUDataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663* (2020).
- [46] Guanyi Mou, Yichuan Li, and Kyumin Lee. 2024. Reducing and Exploiting Data Augmentation Noise through Meta Reweighting Contrastive Learning for Text Classification. (2024).
- [47] Cayque Monteiro Castro Nascimento, Paloma Guimarães Moura, and Andre Silva Pimentel. 2023. Generating structural alerts from toxicology datasets using the local interpretable model-agnostic explanations method. *Digital Discovery* (2023).
- [48] Gang Niu, Marthinus Christoffel du Plessis, Tomoya Sakai, Yao Ma, and Masashi Sugiyama. 2016. Theoretical Comparisons of Positive-Unlabeled Learning against Positive-Negative Learning. In *NeurIPS*.
- [49] Curtis Northcutt, Lu Jiang, and Isaac Chuang. 2017. Learning with confident examples: Rank pruning for robust classification with noisy labels. In *ICML*.
- [50] Ka Hyun Park, Junghun Kim, Jinhong Jung, and U Kang. 2025. PiGLet: Probabilistic Message Passing for Semi-supervised Link Sign Prediction. In *ICDM*.
- [51] Yong-chan Park, Jongjin Kim, and U Kang. 2024. Fast multidimensional partial fourier transform with automatic hyperparameter selection. In *KDD*.
- [52] Yong-chan Park, Kisoo Kim, and U Kang. 2025. PuzzleTensor: a method-agnostic data transformation for compact tensor factorization. In *KDD*.
- [53] Yong-chan Park, SeungJoo Lee, and U Kang. 2026. Fast and Accurate Online Coupled Matrix-Tensor Factorization via Frequency Regularization. *KDD* (2026).
- [54] Giorgio Patrini, Frank Nielsen, Richard Nock, and Marcello Carioni. 2016. Loss factorization, weakly supervised learning and label noise robustness. In *ICML*.
- [55] Harish Ramaswamy, Clayton Scott, and Ambuj Tewari. 2016. Mixture proportion estimation via kernel embeddings of distributions. In *ICLR*.
- [56] Ladislav Rampásek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2022. Recipe for a general, powerful, scalable graph transformer. *NeurIPS* (2022).
- [57] Ekagra Ranjan, Soumya Sanyal, and Partha Talukdar. 2020. Asap: Adaptive structure aware pooling for learning hierarchical graph representations. In *AAAI*.
- [58] Mengye Ren, Wenyuan Zeng, Bin Yang, and Raquel Urtasun. 2018. Learning to Reweight Examples for Robust Deep Learning. In *ICML*.
- [59] Clayton Scott. 2015. A rate of convergence for mixture proportion estimation, with application to learning from noisy labels. In *Artificial Intelligence and Statistics*.
- [60] Nino Shervashidze, Pascal Schweitzer, Erik Jan Van Leeuwen, Kurt Mehlhorn, and Karsten M Borgwardt. 2011. Weisfeiler-Lehman graph kernels. *Journal of Machine Learning Research* (2011).
- [61] Sooyeon Shim, Junghun Kim, Kahyun Park, and U Kang. 2024. Accurate graph classification via two-staged contrastive curriculum learning. *Plos one* (2024).
- [62] Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. 2019. Meta-Weight-Net: Learning an Explicit Mapping For Sample Weighting. In *NeurIPS*.
- [63] Fan-Yun Sun, Jordan Hoffmann, Vikas Verma, and Jian Tang. 2020. InfoGraph: Unsupervised and Semi-supervised Graph-Level Representation Learning via

- Mutual Information Maximization. In *ICLR*.
- [64] Petar Velićković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *ICLR* (2017).
- [65] S Vichy N Vishwanathan, Nicol N Schraudolph, Risi Kondor, and Karsten M Borgwardt. 2010. Graph kernels. *The Journal of Machine Learning Research* (2010).
- [66] Xinrui Wang, Wenhai Wan, Chuanxing Geng, Shaoyuan Li, and Songcan Chen. 2023. Beyond Myopia: Learning from Positive and Unlabeled Data through Holistic Predictive Trends. In *NeurIPS*.
- [67] Zichong Wang, Zhibo Chu, Thang Viet Doan, Shaowei Wang, Yongkai Wu, Vasile Palade, and Wenbin Zhang. 2025. Fair Graph U-Net: A Fair Graph Learning Framework Integrating Group and Individual Awareness. In *AAAI*.
- [68] Oliver Wieder, Stefan Kohlbacher, M  laine Kuenemann, Arthur Garon, Pierre Ducrot, Thomas Seidel, and Thierry Langer. 2020. A compact review of molecular property prediction with graph neural networks. *Drug Discovery Today: Technologies* (2020).
- [69] Man Wu, Shirui Pan, Lan Du, Ivor W. Tsang, Xingquan Zhu, and Bo Du. 2019. Long-short Distance Aggregation Networks for Positive Unlabeled Graph Learning. In *CIKM*.
- [70] Yinjun Wu, Adam Stein, Jacob Gardner, and Mayur Naik. 2023. Learning to Select Pivotal Samples for Meta Re-weighting. In *AAAI*.
- [71] Jiaxing Xu, Qingtian Bian, Xinhang Li, Aihu Zhang, Yiping Ke, Miao Qiao, Wei Zhang, Wei Khang Jeremy Sim, and Balazs Guly  s. 2024. Contrastive Graph Pooling for Explainable Classification of Brain Networks. *IEEE Trans. Medical Imaging* (2024).
- [72] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *ICLR*.
- [73] Hansi Yang, Yongqi Zhang, Quanming Yao, and James T. Kwok. 2023. Positive-Unlabeled Node Classification with Structure-aware Graph Learning. In *CIKM*.
- [74] Nan Yin, Li Shen, Mengzhu Wang, Xiao Luo, Zhigang Luo, and Dacheng Tao. 2023. OMG: Towards Effective Graph Classification Against Label Noise. *IEEE Trans. Knowl. Data Eng.* (2023).
- [75] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform badly for graph representation? *NeurIPS* (2021).
- [76] Zitao Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L. Hamilton, and Jure Leskovec. 2018. Hierarchical Graph Representation Learning with Differentiable Pooling. In *NeurIPS*.
- [77] Jaemin Yoo, Hyunsik Jeon, and U Kang. 2019. Belief Propagation Network for Hard Inductive Semi-Supervised Learning. In *IJCAI*.
- [78] Jaemin Yoo, Saehan Jo, and U Kang. 2017. Supervised belief propagation: Scalable supervised inference on attributed networks. In *ICDM*.
- [79] Jaemin Yoo, Junghun Kim, Hoyoung Yoon, Geonsoo Kim, Changwon Jang, and U Kang. 2021. Accurate graph-based PU learning without class prior. In *International Conference on Data Mining (ICDM)*.
- [80] Jaemin Yoo, Junghun Kim, Hoyoung Yoon, Geonsoo Kim, Changwon Jang, and U Kang. 2022. Graph-based PU learning for binary and multiclass classification without class prior. *Knowl. Inf. Syst.* (2022).
- [81] Jaemin Yoo, Sooyeon Shim, and U Kang. 2022. Model-agnostic augmentation for accurate graph classification. In *WWW*.
- [82] Zaixi Zhang, Qi Liu, Hao Wang, Chengqiang Lu, and Chee-Kong Lee. 2021. Motif-based Graph Self-Supervised Learning for Molecular Property Prediction. In *NeurIPS*.
- [83] Yuchen Zhao, Xiangnan Kong, and S Yu Philip. 2011. Positive and unlabeled learning for graph classification. In *ICDM*.
- [84] Yunrui Zhao, Qianqian Xu, Yangbangyan Jiang, Peisong Wen, and Qingming Huang. 2022. Dist-pu: Positive-unlabeled learning from a label distribution perspective. In *CVPR*.
- [85] Yi Zheng, Hui Peng, Xiaocai Zhang, Zhixun Zhao, Xiaoying Gao, and Jinyan Li. 2019. DDI-PUlearn: a positive-unlabeled learning method for large-scale prediction of drug-drug interactions. *BMC bioinformatics* (2019).
- [86] Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Sch  lkopf. 2004. Learning with local and global consistency. *NeurIPS* (2004).
- [87] Zhangchi Zhu, Lu Wang, Pu Zhao, Chao Du, Wei Zhang, Hang Dong, Bo Qiao, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2023. Robust Positive-Unlabeled Learning via Noise Negative Sample Self-correction. In *KDD*.

A Appendix

A.1 Detailed Settings of Experiments

We provide detailed implementation settings for DELTA-PU and all baseline methods. For baselines that use GIN [72] as the backbone model (e.g., CE, URE, NNRE), we set the number of layers to 5 and the hidden dimension to 64, following standard settings [81]. Since

Facebook does not have node features, we construct them by encoding node degrees as one-hot vectors and use them as input to all methods. Alternatively, one may obtain compressed node features via dimensionality reduction [15–17, 32, 51–53]. Unless a specific pooling method is being evaluated (e.g., AttenPool or GMTPool), we apply mean pooling to aggregate node representations of GIN into a fixed-size graph embedding. Optimization is performed using the Adam optimizer [28] with an initial learning rate of 0.01, which is halved every 50 epochs [81]. All baseline models are trained for 100 epochs, as in previous work [27]. All experiments are conducted on a single machine equipped with an NVIDIA GTX 1080 Ti GPU.

AddPool & MaxPool [11]. Both AddPool and MaxPool use the same GIN encoder as in other baselines, but differ in their global pooling strategies. AddPool aggregates node embeddings of GIN by summing them, while MaxPool takes the element-wise maximum across nodes. We employ the NNRE loss instead of the standard cross-entropy loss, as it offers greater robustness and stability in PU learning scenarios with severe class imbalance.

AttenPool [38]. AttenPool applies an attention-based pooling strategy, where node embeddings are aggregated with learnable attention weights. The attention scores are computed through a gating network that emphasizes informative nodes during graph-level representation. As with other baselines, we use the GIN encoder and apply the NNRE loss instead of the standard cross-entropy loss, due to its improved stability and effectiveness in PU learning settings.

GMTPool [3]. GMTPool produces graph-level embeddings by assigning node representations to multiple learnable prototypes through soft clustering. By leveraging this multi-prototype approach, it effectively captures diverse semantic patterns within a graph, resulting in richer representations compared to conventional single-vector pooling techniques. Consistent with other baselines, we utilize the GIN encoder and adopt the NNRE loss in place of the standard cross-entropy loss, as it provides better stability and performance in PU learning scenarios.

WCE [72]. WCE (weighted cross entropy) is a GIN [72] that uses the binary cross-entropy loss by treating all unlabeled graphs as negative instances. To balance the class imbalance between observed positives and unlabeled graphs, we separately compute the mean loss for positive and unlabeled instances, and then average the two values to form the final loss. We use the GIN code implemented with torch-geometric package [9].

URE [7]. URE minimizes an unbiased risk estimator tailored for PU learning by explicitly modeling the expected loss over positive and unlabeled data. We reimplement URE using the GIN backbone, and assume the class prior π_p is known and fixed throughout training.

NNRE [30]. NNRE extends URE by enforcing non-negativity in the risk function, thereby improving stability in early training and preventing the risk from becoming negative due to estimation variance. We reimplement NNRE using the GIN backbone and assume the class prior is known.

Dist-PU [18, 84] Dist-PU aligns predicted and true label distributions based on known class prior. We reimplement Dist-PU using the GIN backbone. We do not apply the Mixup regularization term, as it is a loss-agnostic technique that can be independently incorporated into other loss functions.

A.2 Graph Classification on Additional Dataset

In Section 4.2, we show that DELTA-PU consistently presents the best performance in diverse real-world datasets. We further evaluate DELTA-PU on an additional synthetic benchmark, BA2Motif [43]. All experimental settings are identical to those described in the main text. As summarized in Table 3, DELTA-PU again outperforms the strong baselines.

Table 3: Graph classification performance on an additional synthetic dataset. The best and second-best performances are highlighted in bold and underlined, respectively. Note that DELTA-PU shows the best performance.

BA2Motif dataset						
Model	$r_{\text{obs}} = 0.1$		$r_{\text{obs}} = 0.1$		$r_{\text{obs}} = 0.1$	
	F1	Acc	F1	Acc	F1	Acc
AddPool	81.30 (5)	81.60 (5)	65.00 (7)	68.70 (7)	70.99 (6)	73.70 (6)
MaxPool	83.43 (3)	83.80 (3)	85.20 (3)	85.60 (3)	71.92 (5)	74.40 (5)
AttenPool	82.68 (4)	83.00 (4)	79.83 (4)	80.90 (4)	69.76 (7)	72.70 (7)
GMTPool	57.90 (8)	65.50 (8)	56.71 (8)	65.40 (8)	66.11 (8)	72.00 (8)
WCE	67.89 (7)	71.70 (7)	70.14 (6)	74.30 (6)	<u>85.02 (2)</u>	<u>85.90 (2)</u>
URE	50.58 (9)	58.50 (9)	55.79 (9)	62.80 (9)	44.81 (9)	56.00 (9)
NNRE	77.60 (6)	78.40 (6)	77.32 (5)	78.00 (5)	79.67 (4)	81.00 (4)
Dist-PU	<u>84.60 (2)</u>	<u>86.60 (2)</u>	<u>88.30 (2)</u>	<u>89.70 (2)</u>	81.08 (3)	83.50 (3)
DELTA-PU (ours)	89.35 (1)	90.10 (1)	93.57 (1)	93.60 (1)	92.16 (1)	93.60 (1)

A.3 Tables

Table 4: Symbols.

Symbol	Description
$\mathcal{G}$	Set of graphs consisting of $\mathcal{G}_{\text{pos}}$ and $\mathcal{G}_{\text{unl}}$
$\mathcal{G}_{\text{pos}}$	Set of observed positive graphs
$\mathcal{G}_{\text{unl}}$	Set of unlabeled graphs that we aim to classify
G_i	i -th graph instance consisting of $\mathcal{V}_i$ and $\mathcal{E}_i$
$\mathcal{V}_i, \mathcal{E}_i$	Sets of nodes and edges in G_i , respectively
π_p	Ground-truth ratio of positive graphs among all graphs
$f_\theta(\cdot)$	Classifier network with parameters θ
$\mathcal{L}(\theta; \mathcal{G})$	Objective function that DELTA-PU aims to minimize
$l(\theta; G_i)$	Instance-wise loss for graph G_i parameterized by θ
$\alpha_i(k)$	Hop-level importance weight of the k -th hop in graph G_i
β_i	Graph-level importance weight for graph G_i

Table 5: Datasets.

Dataset	Graphs	Nodes	Edges	Feat.	Pos.	Neg.
Facebook ¹	995	95,241	101,211	-	448	547
MUTAG ¹	188	3,371	3,721	7	112	76
NCI1 ¹	4,110	122,747	132,753	37	1,852	2,258
NCI109 ¹	4,127	122,494	132,604	38	1,871	2,256
PROTEINS ¹	1,113	43,471	81,044	3	405	708
BAMotifs ²	1,000	25,000	51,392	10	500	500
BAMultiShapes ²	1000	40,000	87,000	10	500	500

¹ <https://chrsmrrs.github.io/datasets/>

² <https://github.com/flyingdoog/PGEexplainer/raw/master/dataset>